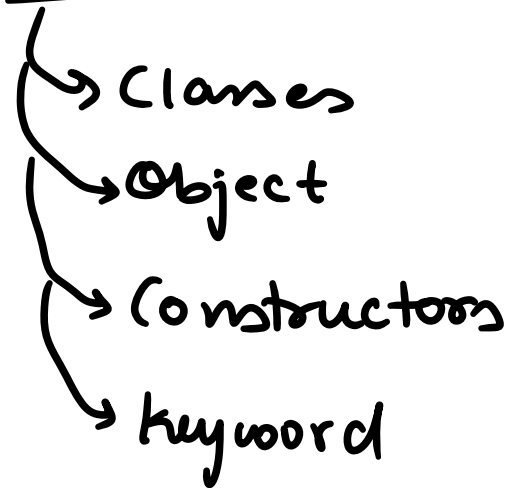


Introduction



Class → It is a named group of property & function

- class start with Capital letter.
- If you want to create your own datatype, you can do it using class.

ex:- data of 5 student with their (roll no, name, marks)

// Create a class.

```
class Student {  
    int rno;  
    string name;  
    float marks;  
}
```

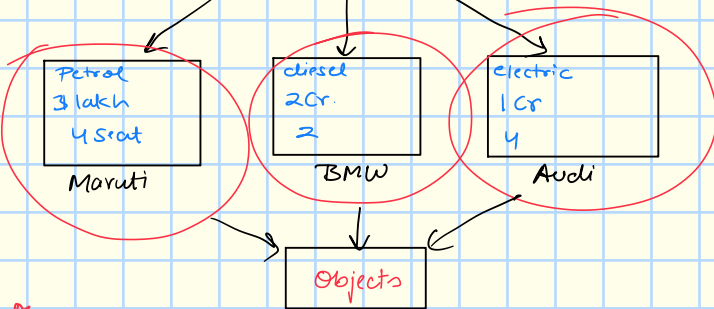
template

Student (Kunal;) → It's datatype is Student
↳ have 3 property

ex:-

class
engine, price, seats

- All car have engine, seat etc → but some have petrol, diesel or electric they have some property but with different values.



class → The architectural blueprint or design template. It specifies what attributes and behaviours the entity will have but contains no data itself.

Instances: The actual physical house built using that blueprint.

ex:- you can use same blueprint to build five distinct houses → each house is a unique instances.

Object & instance mean the exact same physical thing.

Object: is a noun (The entity itself): It focuses on what it is → a self-contained unit holding data & behaviors.

Instances is a relation (It's origin): It focus on where it came from → emphasizing its direct relationship to the class blueprint that created it.

ex:- You are a Person (Object), but you are an instance of humankind (Instance)

(Person example ha humankind ka)

Class is an template of Object & Object is an instance of class

Classes helps us defining these datatypes.

When you creating object of a class, you creating an instance of the class

means physical stuff of that class

ex:- When babies are born they are instances of human class

It is a rule

Class → logical construct

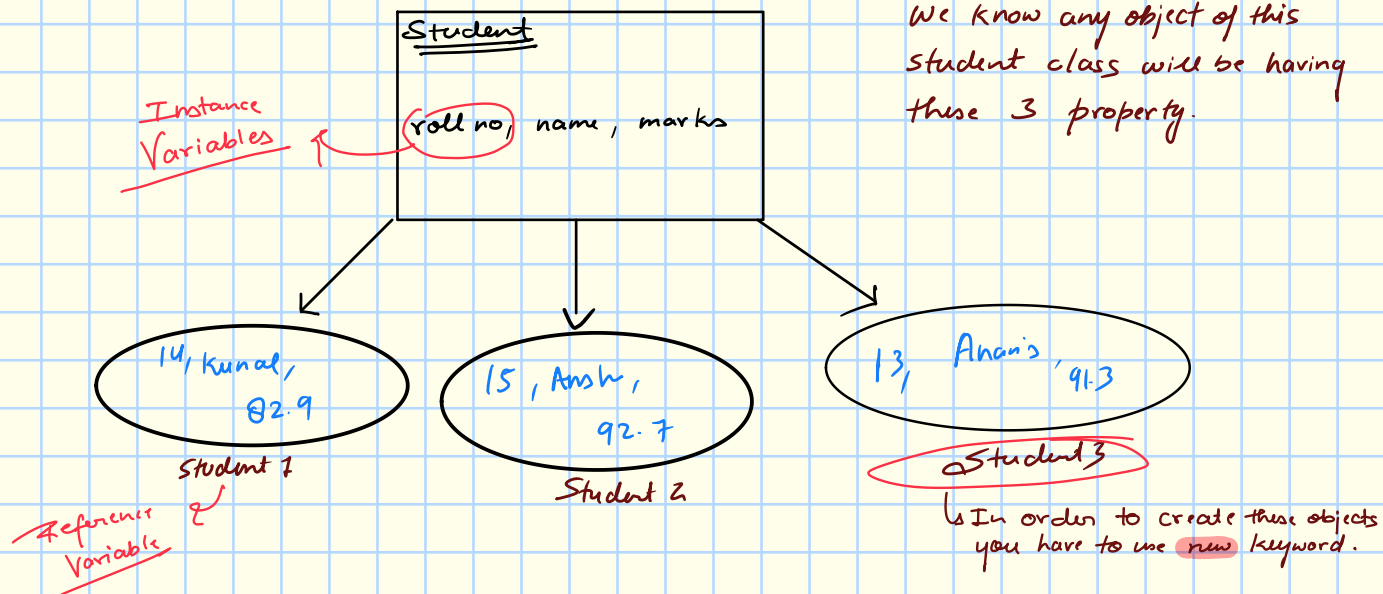
Object → physical reality // occupies space in memory

Objects are characterized by 3 essential properties:

1) **State**: State of an object is a value from its **data type**

2) **Identity**: **Identity** of an object **distinguishes** one object from another.
→ It is useful to think of an object's identity as the place where its value is stored in memory.

3) **behaviour**: The behaviour of an object is the effect of data-type operation.



⇒ How to access instance variable?

→ Using dot(.) operator.

`cout << Student 1 . roll no`
→ 14

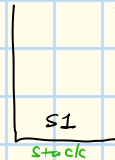
Instance Variable

• All variable inside your object are instance variable

```
class Student {
    int rno;
    string name;
    float marks;
}
```

`Student s1;` // declaring a object
→ not creating a object

⇒ (basically it is just a reference to an object of type student)

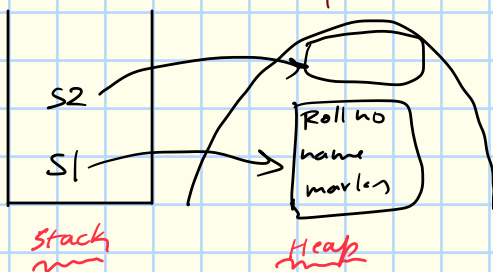


⇒ To create object

`Student s1 = new Student();`

(allocates at runtime memory)

dynamically allocates memory for an object & returns a reference to it.



runtime → name suggest program is running. that is when memory allocated that is known as **dynamic memory allocation**

Student s1 = new Student();

Compile-time

Runtime

```
class Student {  
    int rno;  
    String name;  
    float marks;  
}
```

Student s1 = new Student(); // declared in main class
System.out(s1.rno); → 0
System.out(s1.name); → null
System.out(s1.marks); → 0.0

Acc. to this class

⇒ Student s1 = new Student();

s1.rno = 14;
s1.name = "Amshul";
s1.marks = 84.4f;

System.out(s1.rno); → 14
System.out(s1.name); → Amshul
System.out(s1.marks); → 84.4

⇒ Working

Using new
I create a
object

Let's its reference variable
name is Kunal.

new

Student

rno = 0
name = null
marks = 0.0

Kunal

rno = 14
name = "Kunal Kush"

kunal.rno; →

print
0

kunal.rno = 14; → 14

kunal.name = "Kunal Kush" → Kunal Kush

→ if we try to print kunal.marks

↳ then check in its object
if there is no marks present
so, print default value.

Rahul

rahul.rno →

print
0

rahul.name → null

rahul.marks → 0.0

no value is defined here it printed
default values.

⇒ Box mybox; // declare reference to object
mybox = new Box(); // allocates a box object.

→ The first line declares mybox as a reference to an object of type Box.

→ At this point, mybox does not yet refer to an actual object.

After second line executes, you can use mybox as if it were Box object. But, in reality, mybox simply holds, in essence, the memory address of the actual Box objects.

Constructor

Student s1 = new Student(); → constructor

Constructor is special function that runs when you create an object & it allocates some variable.

• Student s1 = new Student(13, "Kunal", 84.4f);

→ bind these arguments with objects
→ special type of function in the class

⇒ this

```
class Student {  
    int rno;  
    String name;  
    float marks;
```

* no need to add return type
name → It's return type is it's class itself
↳ not required

no. of parameters none
hence, it will find
constructor with none
parameter

Student() {

this.rno = 13;

this.name = "Kunal Kush";

this.marks = 84.4;

→ constructor

Student kunal = new Student();

this will be replaced by kunal
& it'll seem like

kunal.rno = 13;

kunal.name = "Kunal Kush"

kunal.marks = 84.4

⇒ rno, name,
marks
get the value
which is
assigned inside
constructor

this is nothing → It's just replacing
(this) with the name of the reference
variable itself.

⇒ Constructor without parameter

```
class Student {  
    int rno;  
    String name;  
    float marks;
```

```
    void greetings() {  
        cout ("Hello! My name is" + this.name);  
    }
```

```
    void changename(String new name) {  
        this.name = new name;  
    }
```

```
student () {
```

```
    this.rno = 14;
```

```
    this.name = "Kunal Kush";
```

```
    this.marks = 84.4f;
```

```
}
```

```
}
```

After creating
object with
reference s1

s1.rno = 14
s1.name = "Kunal Kush"
s1.marks = 84.4f

How it
seems like

→ this replaced with s1

```
Student s1 = new Student();
```

```
→ s1.greetings();
```

// Hello! My name is Kunal Kush

```
→ s1.change name (Anshul);
```

```
→ s1.greetings ();
```

// Hello! My name is Anshul

```
student () {
```

```
    this.rno = 14;
```

```
    this.name = "Kunal Kush";
```

```
    this.marks = 84.4f;
```

```
}
```

```
}
```

but the property will be same for every
object.

⇒ Constructor with parameters

```
Student (int rno, String name, float marks) {
```

```
    this.rno = rno;
```

```
    this.name = name;
```

```
    this.marks = marks;
```

```
}
```

Constructors
with 3 parameters

Internally this is like.

s1.marks = marks

↓
whatever you putting
through the constructor

→ if we not use (this) here then it will
print default value, bhe hi humne values
daal di ho constructor me.

```
Student s1 = new Student (15, "Anakoji", 50.0);
```

```
System.out (s1.rno); → 15
```

```
System.out (s1.name); → Anakoji
```

```
System.out (s1.marks); → 50.0
```

* s1 replace with (this)

this.marks = marks

Constructor Overloading

feature that allows a class to define multiple constructor with the same name but diff. parameter lists.

```
student () {  
    this.name = "anshul";  
    this.rno = 19;  
}  
  
student (String name, int rno) {  
    this.name = name;  
    this.rno = rno;  
}  
  
student (String name, int rno, float marks) {  
    this.name = name;  
    this.rno = rno;  
    this.marks = marks;  
}
```

without parameter

with parameter but with 2

with 3 parameter

rno → 14
name → Kunal Kush
marks → 88.4f

⇒ Constructor takes value from other object

```
Student (Student other) {  
    this.name = other.name;  
    this.rno = other.rno;  
    this.marks = other.marks;  
}
```

will be replaced with random

will be replaced with Kunal

Student Kunal = new Student();

Student random = new Student (Kunal);
Sout(random.name); // Kunal Kush

Object passed

⇒ It'll be seen like

random.name = Kunal.name.

⇒ How to call constructor from another constructor?

```
Student () {  
    // this is how you call constructor from another constructor.  
    this (13, "Anshul", 100.0f);  
}
```

```
student (int rno, String name, float marks) {  
    this.rno = rno;  
    this.name = name;  
    this.marks = marks;  
}
```

Student random = new student();

Why we don't use "new" keyword for creating primitive data types?

↳ because they are implemented as like normal variable, we don't implement those as objects, they are just like primitive datatypes, we do this to increase efficiency to make it more fast.

In java primitive data types are not implemented as Objects

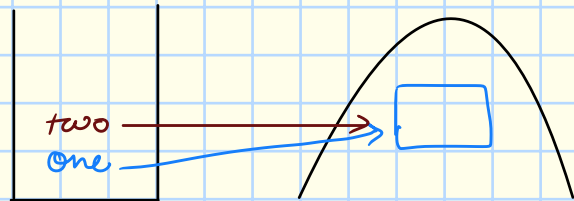
→ Objects are stored in heap memory, but in java primitives are not object. Hence, they are only stored in stack memory.

Memory allocation of "new" keyword

```
Student one = new Student;  
Student two = one;
```

∴ two & one both pointing to same object

→ if change made by any (one or two) it will reflected for other also.



⇒ Code

```
Student one = new Student;  
Student two = one;
```

```
one.name = "Something"; # change made via (one)  
Sout(two.name);         # but also reflected in (two)
```

Wrapper class (Java is pass by value)

In Java there is no such thing pass by reference

Integer num = 45; // num is object of type Integer.

Code

```
public class Wrapper {  
    psvm(...) {  
        int a = 10;  
        int b = 20;  
  
        swap(a, b);  
        Sout(a + " " + b); // 10 20  
    }  
  
    static void swap(int a, int b) {  
        int temp = a;  
        a = b;  
        b = temp;  
    }  
}
```

It not swapped

actually swap happen in this scope only

```
public class Wrapper {  
    psvm(...) {  
        Integer a = 10;  
        Integer b = 20;  
  
        swap(a, b);  
        Sout(a + " " + b); // 10 20  
    }  
  
    static void swap(int a, int b) {  
        int temp = a;  
        a = b;  
        b = temp;  
    }  
}
```

Still not change as a & b are object

broz of final

Final

- It is a keyword.
- Can prevent from content to be modified.
- Can make like constant.
- Variable name should be in Capital.
- Always initialized while declaring.

ex:-

```
final int BONUS = 2;
```

// bonus = 3; → will give error as can't change.

} In primitive value
cannot be change once
initialize

⇒ In non-primitive value can change but cannot reassign.

```
final student s1 = new student();
```

```
s1.name = "new-name";
```

```
s1.name = new student(); X
```

Garbage Collection

↳ (It does automatically in Java)

- We cannot destroy object manually, but can perform some task when Java will destroy.

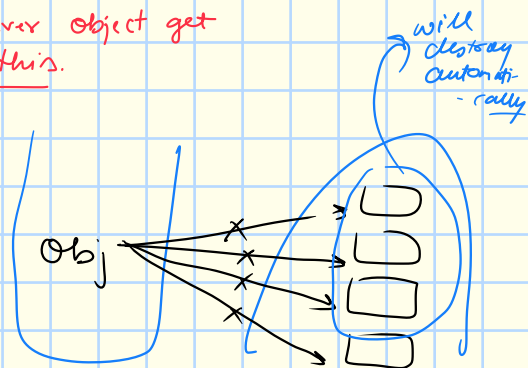
Use finalize() to do this

ex:-

```
class A {  
    ;  
    public A( ... ) {  
        ;  
    }  
    @Override  
    protected void finalize() throws Throwable {  
        System.out.println("Object is destroyed");  
    }  
}  
public class main {  
    public static void main( ... ) {
```

```
        A obj;  
        for(int i=0; i<1000000000; i++) {  
            obj = new A("random name");  
        }  
    }  
}
```

↳ Through this whenever object get
destroy will print this.



Imp

Instance → An instance is what you get when you instantiate a class using the new keyword.

each instance has →

- its own memory location
- A unique identity.
- independent instance variable.
- Access to instance method

ex:- Car car1 = new Car("Tesla", 2021); // instance 1
 Car car2 = new Car("Ford", 2020); // instance 2

NOTE: car1, car2 are instances of the Car class.
 Each has its own model and year.

Instance v/s object

In java, the terms object & instance are often used interchangeably broz every object is an instance of class.

Object is more general term referring to any runtime entity.

Instance emphasizes the relationship to a specific class ("an instance of car")

What Is an Object?

An **object** in Java is a runtime entity that encapsulates **state** (data/fields) and **behavior** (methods). 📖 It is the physical manifestation of a class blueprint, allocated in heap memory.

How Objects Differ From Each Other

Even if two objects belong to the same class and hold identical data, they are distinct entities. They differ in three fundamental ways:

- 1. Identity (Memory Location)**

Every object has a unique identity determined by its specific location in the heap memory. 📖

 - **Reference Equality (==):** Checks if two references point to the **exact same** memory address. 📖
 - **Identity hashCode:** The default `hashCode()` (or `System.identityHashCode()`) returns a value derived from this memory address, ensuring every distinct object has a unique ID unless overridden. 📖
- 2. State (Instance Variables)**

Each object maintains its **own copy** of instance variables. 📖 Changing the state of one object does not affect another, even if they were created from the same class.

```
Car car1 = new Car("Tesla", 2024);
Car car2 = new Car("Tesla", 2024);

// car1 and car2 have the SAME state (data)
// but DIFFERENT identity (memory addresses)
System.out.println(car1 == car2); // false
```
- 3. Behavior via State**

While the **code** for methods is shared (stored in the method area), the **execution** of instance methods operates on the specific state of that particular object. 📖

How Objects Are Created

Objects are primarily created using the **new** keyword, which performs three critical steps:

- 1. Declaration:** Defines a reference variable (e.g., `Car myCar;`). No memory is allocated yet. 📖
- 2. Instantiation:** The **new** operator allocates memory for the object in the **heap**. 📖
- 3. Initialization:** The class **constructor** is called to initialize the object's state. 📖

```
Car myCar = new Car("Tesla", 2024);
```

While **new** is the standard method (used in ~99% of cases), Java supports four other ways to create objects:

- **Reflection:** Using `Class.forName().newInstance()` or `Constructor.newInstance()`. 📖
- **Cloning:** Using the `clone()` method to copy an existing object (does not call the constructor). 📖
- **Deserialization:** Reading an object from a byte stream (e.g., a file), which reconstructs the object without invoking a constructor. 📖

Feature	Object A	Object B
Class Type	Car	Car
Memory Address	0x1234 (Unique)	0x5678 (Unique)
State (Data)	model="Tesla"	model="Tesla"
Identity	Distinct	Distinct

Feature	Object A	Object B
Class Type	Car	Car
Memory Address	0x1234 (Unique)	0x5678 (Unique)
State (Data)	model="Tesla"	model="Tesla"
Identity	Distinct	Distinct

- **Object:** Refers to the **entity itself** (the data and behavior in memory).
 - *Context:* "I created an **object** to store the car data."
- **Instance:** Refers to the **relationship** between that object and its class.
 - *Context:* "This object is an **instance** of the **Car** class."

The Relationship

Think of it like a **specific house** and a **building**:

- The code `new Car(...)` builds a **house** (the **object**).
- Because that house was built from the "House Blueprint," it is also an **instance** of a house.

In your code:

```
Car car1 = new Car("Tesla", 2024);
```



1. `new Car(...)` creates the **object** in memory.
2. Because it was created from the **Car** class, it is technically an **instance** of **Car**.
3. `car1` is the **reference** (variable) pointing to it.

Summary: Every **instance** is an **object**, and every **object** (created from a class) is an **instance**. The distinction is only grammatical, not technical.

When we create an object in Java, we create an instance of class that has its own set of properties & can perform action based on the behaviour.

